

HỌC VIỆN NGÂN HÀNG
KHOA HỆ THỐNG THÔNG TIN QUẢN LÝ

BÁO CÁO TIỂU LUẬN

Học phần: Biểu diễn tri thức và ứng dụng (Cao học)

Hệ thống hỗ trợ phân tích và chẩn đoán sự cố hệ thống mạng và dịch vụ CNTT sử dụng Knowledge Graph, GraphRAG và Rule-Based Reasoning

Học viên thực hiện: **Khang**

Giảng viên hướng dẫn: Giảng viên hướng dẫn học phần

Ngày 18 tháng 7 năm 2026

Mục lục

Tóm tắt	3
1 Giới thiệu	4
1.1 Lý do chọn đề tài	4
1.2 Mục tiêu đề tài	4
1.3 Đối tượng và phạm vi nghiên cứu	5
2 Cơ sở lý thuyết	6
2.1 Biểu diễn tri thức bằng Knowledge Graph	6
2.2 Suy diễn dựa luật (Rule-Based Reasoning)	6
2.3 GraphRAG	7
2.4 Kết hợp luật và GraphRAG: kiểm chứng chéo	7
3 Thiết kế và kiến trúc hệ thống	8
3.1 Kiến trúc tổng thể	8
3.2 Schema Knowledge Graph	9
3.3 Tổng quát hoá sang đồ thị phụ thuộc đa-cha (DAG)	10
4 Cài đặt chi tiết	12
4.1 Bộ luật suy diễn (Rule Engine)	12
4.2 Pipeline GraphRAG	13
4.3 Module tổng hợp và cơ chế kiểm chứng chéo	14
5 Dữ liệu: hành trình tìm và tích hợp dữ liệu thật	16
5.1 Vì sao cần tìm dữ liệu thật	16
5.2 Ba nguồn được khảo sát	16
5.3 Dữ liệu NetManAIOps/DejaVu	17
6 Giao diện demo	19
7 Kết quả thực nghiệm và đánh giá	20
7.1 Phương pháp đánh giá	20

7.2	Kết quả trên tập viển thông mô phỏng	20
7.3	Kết quả trên 78 sự cố thật DejaVu	21
7.4	Thử nghiệm với LLM thật và phát hiện về hiệu chỉnh độ tin cậy	21
7.5	Phân tích lỗi có hệ thống trên mẫu mở rộng (Error Analysis)	22
7.6	Kiểm thử phần mềm	24
8	Kết luận và hướng phát triển	25
8.1	Kết luận	25
8.2	Hướng phát triển	25

Tóm tắt

Báo cáo trình bày thiết kế, cài đặt và đánh giá thực nghiệm của một hệ thống hỗ trợ chẩn đoán nguyên nhân gốc rễ (root cause) sự cố trên hệ thống mạng và dịch vụ CNTT, kết hợp ba kỹ thuật biểu diễn và suy diễn tri thức đã học trong học phần: **Knowledge Graph (KG)** để biểu diễn tường minh cấu trúc phụ thuộc giữa thiết bị/dịch vụ, **suy diễn dựa luật (rule-based reasoning)** theo mô hình production system cổ điển với cơ chế kết hợp độ tin cậy kiểu MYCIN (certainty factor), và **GraphRAG** (Retrieval-Augmented Generation trên đồ thị tri thức) để khai thác một mô hình ngôn ngữ lớn (LLM) sinh giải thích dựa trên ngữ cảnh được xác thực từ đồ thị thay vì suy đoán tự do.

Hệ thống được cài đặt đầy đủ bằng Python, dùng Neo4j làm hạ tầng lưu trữ đồ thị tri thức, và được đánh giá trên hai tập dữ liệu độc lập: (1) một tập dữ liệu mô phỏng mạng viễn thông gồm 9 thiết bị, 2 site và 5 kịch bản sự cố mẫu, đúng phạm vi đã đăng ký trong đề cương; và (2) một tập **dữ liệu thật** gồm 78 sự cố hạ tầng IT-ops/microservice thật, lấy từ bộ dữ liệu công khai NetManAIOps/DejaVu, dùng để kiểm chứng độc lập khả năng tổng quát hoá của phương pháp trên một hệ thống có cấu trúc phụ thuộc thực tế mà nhóm không tự tạo ra. Trên cả hai tập dữ liệu, cả ba phương án — chỉ dùng luật, chỉ dùng GraphRAG, và kết hợp cả hai — đều đạt độ chính xác top-1 100% với LLM mô phỏng (MockLLM). Khi thay bằng một LLM thật (Groq/Llama 3.3-70B), báo cáo ghi nhận và phân tích một phát hiện thực nghiệm quan trọng: LLM thật có thể tự tin nhưng sai, và module tổng hợp cần được thiết kế lại để không cho một câu trả lời GraphRAG không được luật nào xác nhận lẫn át một câu trả lời đúng từ Rule Engine — đúng minh hoạ cho vai trò "kiểm chứng chéo" của suy diễn dựa luật mà lý thuyết của học phần đề cập.

Chương 1

Giới thiệu

1.1 Lý do chọn đề tài

Trong vận hành mạng viễn thông, kỹ sư trung tâm điều hành mạng (NOC – Network Operations Center) hằng ngày phải xử lý một khối lượng lớn cảnh báo (alarm), chỉ số hiệu năng (KPI) và log thiết bị đến từ nhiều lớp mạng khác nhau (truyền dẫn, truy nhập, core, dịch vụ). Việc xác định nguyên nhân gốc rễ của sự cố hiện chủ yếu dựa vào kinh nghiệm cá nhân và tra cứu tài liệu rời rạc, dẫn đến thời gian xử lý sự cố (MTTR) kéo dài và khó chia sẻ tri thức giữa các kỹ sư.

Bài toán này phù hợp để áp dụng các kỹ thuật biểu diễn tri thức đã học: tri thức về cấu trúc mạng, quan hệ phụ thuộc giữa thiết bị và dịch vụ, cũng như kinh nghiệm xử lý sự cố có thể được mô hình hoá tường minh dưới dạng Knowledge Graph; việc suy luận nguyên nhân có thể được hỗ trợ bởi cả suy diễn dựa luật và truy xuất ngữ cảnh bằng GraphRAG kết hợp LLM. Đây đồng thời là một bài toán kinh điển trong lĩnh vực AIOps (AI for IT Operations), nên ngoài mạng viễn thông, phương pháp còn được kiểm chứng thêm trên dữ liệu hạ tầng CNTT/microservice thật (Mục 5), cho thấy tính tổng quát của cách tiếp cận.

1.2 Mục tiêu đề tài

Mục tiêu tổng quát: Xây dựng hệ thống hỗ trợ kỹ sư vận hành phân tích, chẩn đoán nguyên nhân gốc rễ của sự cố và đề xuất hướng xử lý dựa trên tri thức được biểu diễn dưới dạng đồ thị tri thức.

Mục tiêu cụ thể:

- (1) Thiết kế mô hình Knowledge Graph biểu diễn thiết bị, cảnh báo, KPI và các mối quan hệ (kết nối vật lý/logic, phụ thuộc dịch vụ, nguyên nhân – hệ quả).
- (2) Xây dựng bộ luật suy diễn (rule-based reasoning) mã hoá kinh nghiệm chẩn đoán sự cố của kỹ sư vận hành dưới dạng luật IF-THEN (production rules).

- (3) Xây dựng pipeline GraphRAG: khi có sự cố, truy xuất các thực thể và quan hệ liên quan trong KG, kết hợp ngữ cảnh này với LLM để sinh giải thích và đề xuất nguyên nhân.
- (4) Kết hợp kết quả suy diễn luật và kết quả truy xuất GraphRAG thành khuyến nghị nguyên nhân gốc rễ và hướng xử lý cho kỹ sư vận hành, với cơ chế kiểm chứng chéo.
- (5) Xây dựng demo/prototype minh họa toàn bộ luồng xử lý, gồm cả giao diện web, trên một tập kịch bản sự cố mẫu và một tập dữ liệu sự cố thật độc lập.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: các phương pháp biểu diễn tri thức bằng đồ thị (Knowledge Graph), kỹ thuật Retrieval-Augmented Generation trên đồ thị (GraphRAG), và các phương pháp suy diễn dựa luật (rule-based/production system).

Phạm vi dữ liệu gồm hai lớp:

- *Lớp chính:* dữ liệu mô phỏng/giả lập về topology mạng viễn thông, cảnh báo và KPI ở quy mô vừa phải (một số loại thiết bị tiêu biểu: BTS/gNodeB, OLT, router, thiết bị truyền dẫn), do giới hạn thời gian của học phần không đi sâu vào việc kết nối dữ liệu vận hành thực tế đang bảo mật.
- *Lớp đối chiếu bổ sung:* dữ liệu sự cố **thật** ngoài miền viễn thông (hạ tầng IT-ops/microservice, bộ dữ liệu công khai NetManAIOps/DejaVu, giấy phép CC-BY 4.0), dùng để kiểm chứng độc lập khả năng tổng quát hoá của phương pháp trên một hệ thống có cấu trúc phụ thuộc thực tế, không tự tạo dữ liệu (xem Mục 5).

Phạm vi kỹ thuật: tập trung vào việc mô hình hoá tri thức, xây dựng bộ luật suy diễn và luồng GraphRAG kết hợp LLM; không đi sâu vào tối ưu hạ tầng triển khai production.

Chương 2

Cơ sở lý thuyết

2.1 Biểu diễn tri thức bằng Knowledge Graph

Tri thức về hệ thống mạng được mô hình hoá dưới dạng đồ thị có hướng gồm các thực thể (node) và quan hệ (edge). Mô hình dữ liệu được tổ chức theo hướng ontology đơn giản (class – property – instance), thuận lợi cho việc truy vấn quan hệ nhiều bước (multi-hop) khi chẩn đoán sự cố lan truyền qua nhiều lớp mạng. Sáu lớp thực thể được định nghĩa: *Device*, *Alarm*, *KPI*, *Service*, *Site*, *Incident*; và tám loại quan hệ: *CONNECTS_TO*, *DEPENDS_ON*, *BELONGS_TO_SITE*, *RAISED_ALARM*, *AFFECTS_SERVICE*, *CAUSED_BY*, *HAS_KPI*, *PART_OF_INCIDENT*.

Một điểm quan trọng được rút ra trong quá trình cài đặt (Mục 3.3): quan hệ *DEPENDS_ON* không nên bị ràng buộc thành một cây đơn-cha (mỗi thiết bị chỉ có một cha), mà cần biểu diễn như một **đồ thị có hướng không chu trình (DAG)** cho phép nhiều-cha, vì đây mới là cấu trúc phụ thuộc thực tế của cả mạng viễn thông (dự phòng, dual-homing) lẫn hạ tầng microservice (một dịch vụ gọi nhiều backend và đồng thời phụ thuộc vào máy chủ vật lý nó chạy trên).

2.2 Suy diễn dựa luật (Rule-Based Reasoning)

Suy diễn dựa luật được cài đặt theo mô hình *production system* kinh điển: mỗi luật có dạng **IF (điều kiện trên các sự kiện quan sát được trong KG) THEN (giả thuyết nguyên nhân, kèm độ tin cậy)**. Khi nhiều luật cùng chỉ ra một thiết bị, độ tin cậy được kết hợp theo công thức *certainty factor* của hệ chuyên gia MYCIN [1]:

$$CF_{1,2} = CF_1 + CF_2 \times (1 - CF_1) \quad (2.1)$$

Công thức (2.1) có hai tính chất phù hợp với bài toán: (i) đơn điệu tăng — càng nhiều bằng chứng độc lập ủng hộ cùng một giả thuyết thì độ tin cậy tổng hợp càng cao; (ii) bị chặn trong $[0, 1]$, tránh hiện tượng độ tin cậy vượt ngưỡng khi kết hợp nhiều nguồn.

2.3 GraphRAG

GraphRAG (Retrieval-Augmented Generation trên đồ thị tri thức) là kỹ thuật kết hợp truy xuất thông tin có cấu trúc từ một Knowledge Graph với khả năng sinh văn bản của LLM, nhằm hạn chế hiện tượng LLM "ảo giác" (hallucination) khi suy luận trên tri thức chuyên biệt. Quy trình gồm ba bước: (1) xác định các thực thể liên quan đến sự cố (thiết bị, cảnh báo, KPI bất thường) — gọi là *seed entities*; (2) truy xuất tiểu đồ thị (subgraph) đa bước quanh các seed entities này, bao gồm cả các sự cố lịch sử tương tự; (3) chuyển tiểu đồ thị và ngữ cảnh liên quan thành văn bản có cấu trúc để đưa vào LLM, giúp LLM sinh giải thích/nguyên nhân dựa trên tri thức đã được xác thực thay vì suy đoán tự do.

2.4 Kết hợp luật và GraphRAG: kiểm chứng chéo

Vì rule-based reasoning dựa trên tri thức tường minh, đã kiểm chứng, còn GraphRAG dựa trên suy luận xác suất của một mô hình thống kê (LLM), hai phương pháp bổ trợ lẫn nhau: luật thu hẹp không gian nguyên nhân một cách chắc chắn, còn LLM có khả năng diễn giải các mẫu hình chưa được luật hoá tường minh (ví dụ so khớp với sự cố lịch sử tương tự). Mục 7 trình bày một phát hiện thực nghiệm cho thấy vai trò "kiểm chứng chéo" này không chỉ là lý thuyết suông mà thực sự cần thiết khi làm việc với LLM thật.

Chương 3

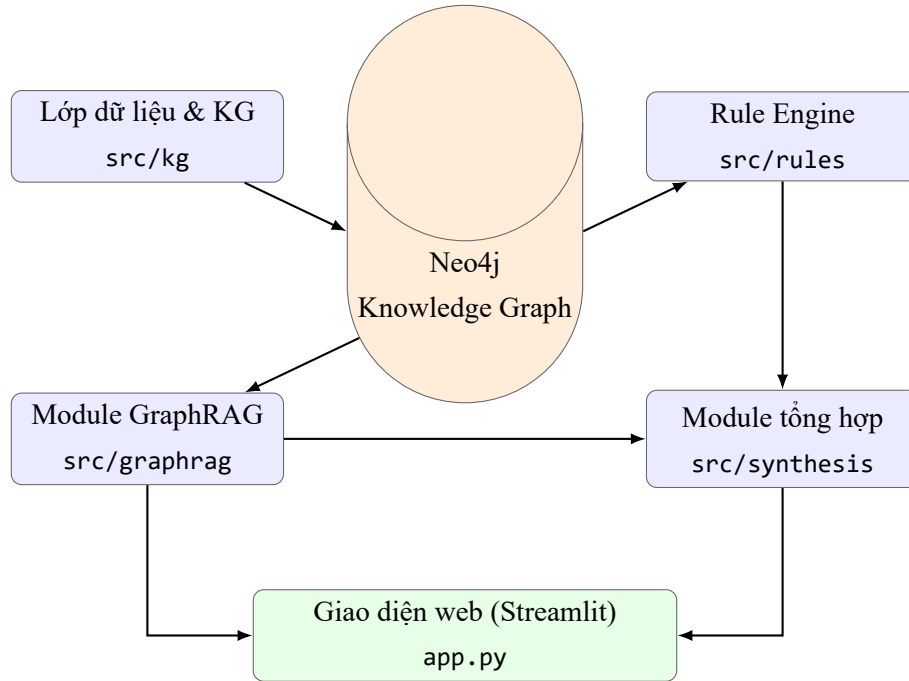
Thiết kế và kiến trúc hệ thống

3.1 Kiến trúc tổng thể

Hệ thống gồm bốn thành phần chính, tương ứng bốn package Python (`src/kg`, `src/rules`, `src/graphrag`, `src/synthesis`), giao tiếp qua một cơ sở dữ liệu đồ thị Neo4j dùng chung (Hình 3.1):

- (1) **Lớp dữ liệu và Knowledge Graph** (`src/kg`): định nghĩa schema, nạp dữ liệu thiết bị/cảnh báo/KPI/quan hệ mạng vào Neo4j.
- (2) **Module suy diễn luật – Rule Engine** (`src/rules`): áp dụng các luật IF-THEN để đề xuất tập nguyên nhân ứng viên ban đầu.
- (3) **Module GraphRAG** (`src/graphrag`): truy xuất tiêu đồ thị liên quan, sinh giải thích và đề xuất nguyên nhân bằng LLM dựa trên ngữ cảnh truy xuất được.
- (4) **Module tổng hợp – Synthesis** (`src/synthesis`): kết hợp kết quả từ hai module trên, xếp hạng nguyên nhân theo độ tin cậy, đề xuất hướng xử lý.

Bên trên bốn thành phần lõi là một giao diện web (`app.py`, Chương 6) và hai kịch bản dùng lại toàn bộ pipeline nhưng trên hai nguồn dữ liệu khác nhau: kịch bản demo/đánh giá viễn thông (`src/demo.py`, `src/evaluation/evaluate.py`) và kịch bản đánh giá trên dữ liệu thật DejaVu (`src/evaluation/evaluate_dejavu.py`).



Hình 3.1: Kiến trúc tổng thể của hệ thống: bốn module lõi giao tiếp qua Neo4j, tổng hợp kết quả lên giao diện web.

3.2 Schema Knowledge Graph

Bảng 3.1 tóm tắt schema KG. Toàn bộ ràng buộc UNIQUE trên thuộc tính id của mỗi nhân được khai báo bằng *constraint* của Neo4j, đảm bảo tính toàn vẹn khi nạp lại dữ liệu nhiều lần.

Bảng 3.1: Schema Knowledge Graph

Thành phần	Mô tả
<i>Nhãn thực thể (node label)</i>	
Device	Thiết bị mạng/hạ tầng (transmission, router, olt, gnodeb, docker, db, os, osb...)
Alarm	Một cảnh báo cụ thể, gắn với thiết bị phát sinh (type, severity)
KPI	Một chỉ số đo lường bất thường (name, value, status)
Service	Dịch vụ nghiệp vụ bị ảnh hưởng (Mobile Data, Broadband...)
Site	Vị trí vật lý của thiết bị
Incident	Một sự cố (đang xảy ra hoặc lịch sử), gộp các Alarm/KPI liên quan
<i>Quan hệ (relationship type)</i>	
DEPENDS_ON	Thiết bị con phụ thuộc thiết bị cha (đồ thị DAG, cho phép nhiều cha)
CONNECTS_TO	Kết nối vật lý/logic giữa hai thiết bị
RAISED_ALARM	Thiết bị → cảnh báo mà nó phát sinh
HAS_KPI	Thiết bị → chỉ số KPI đo được
AFFECTS_SERVICE	Thiết bị → dịch vụ bị ảnh hưởng nếu thiết bị lỗi
BELONGS_TO_SITE	Thiết bị → site vật lý
PART_OF_INCIDENT	Alarm/KPI → sự cố đang xét
CAUSED_BY	Incident → thiết bị là nguyên nhân gốc rễ (ground truth)

3.3 Tổng quát hoá sang đồ thị phụ thuộc đa-cha (DAG)

Thiết kế ban đầu giả định mỗi thiết bị chỉ phụ thuộc duy nhất một thiết bị cha (`depends_on` là một giá trị đơn), phù hợp với topology hình cây của tập dữ liệu viễn thông mô phỏng. Khi tích hợp dữ liệu thật DejaVu (Chương 5), topology thực tế cho thấy một container có thể đồng thời gọi nhiều container/database khác và phụ thuộc vào máy chủ vật lý nó chạy trên — tức là **hiều-cha**. Toàn bộ Rule Engine, module truy xuất subgraph của GraphRAG và module suy diễn của MockLLM đã được tổng quát hoá: `parent_id` (một giá trị) được thay bằng `parents_of[device]` (một danh sách), các luật liên quan đến "thiết bị cha" được viết lại để kiểm tra *có tồn tại một cha nào bị cảnh báo hay không* thay vì so sánh với một cha duy nhất. Đây là một tổng quát hoá hợp lý ngay cả với mạng viễn thông thật (một router có thể có hai đường uplink dự phòng), không phải một giải pháp vá lỗi riêng cho dữ liệu DejaVu. Toàn

bộ 5 kịch bản viễn thông vẫn cho kết quả đúng 100% sau khi tổng quát hoá (Bảng 7.1), xác nhận thay đổi không phá vỡ hành vi cũ.

Chương 4

Cài đặt chi tiết

4.1 Bộ luật suy diễn (Rule Engine)

Bốn luật được xây dựng, mã hoá các mẫu hình chẩn đoán phổ biến trong vận hành NOC (Bảng 4.1). Việc lựa chọn bốn mẫu hình này dựa trên phân tích cách một kỹ sư NOC thường suy luận: từ mẫu hình lan truyền theo cấu trúc (luật R1, R3) đến mẫu hình khu trú (luật R2) và mẫu hình cảnh báo sớm dựa trên xu hướng KPI (luật R4).

Bảng 4.1: Bốn luật IF-THEN của Rule Engine

Luật	Điều kiện (IF) → Kết luận (THEN)	CF
R1: Cascading parent failure	IF thiết bị X phát cảnh báo mất kết nối (Link_Down) VÀ toàn bộ thiết bị con của X đều phát cảnh báo mất tín hiệu THEN nguyên nhân gốc rễ là X	0.95
R2: Isolated device fault	IF thiết bị X phát cảnh báo VÀ không có thiết bị cha nào bị cảnh báo VÀ không có thiết bị con nào bị cảnh báo THEN nguyên nhân gốc rễ là X	0.85
R3: Shared-parent sibling pattern	IF ≥ 2 thiết bị con cùng một cha P đều phát cảnh báo VÀ P chưa có cảnh báo tường minh THEN P là điểm lỗi chung	0.75
R4: KPI-only early warning	IF thiết bị X có KPI vượt ngưỡng cảnh báo/nguy hiểm VÀ chưa có alarm chính thức THEN nghi ngờ X đang suy giảm hiệu năng	0.50–0.65

Luật R1 chính là ví dụ đã nêu trong đề cương ban đầu ("NẾU thiết bị truyền dẫn X mất kết nối VÀ tất cả thiết bị con của X đều phát cảnh báo mất tín hiệu THÌ nguyên nhân gốc rễ

hiều khả năng là sự cố tại X”). Khi nhiều luật cùng chỉ ra một thiết bị, độ tin cậy được kết hợp bằng công thức (2.1) (Đoạn mã 4.1).

```

1 def _combine_certainty_factors(confidences):
2     """MYCIN-style certainty-factor combination: CF12 = CF1 + CF2*(1-CF1)."""
3     combined = 0.0
4     for c in confidences:
5         combined = combined + c * (1 - combined)
6     return round(combined, 4)

```

Listing 4.1: Kết hợp độ tin cậy kiểu MYCIN (src/rules/rule_engine.py)

Đoạn mã 4.2 minh hoạ cách cài đặt luật R1 bằng Python thuần, thao tác trên các cấu trúc dữ liệu (devices, children_of, alarms_by_device) được truy vấn một lần từ Neo4j (hàm fetch_facts) rồi đánh giá lại nhiều lần trong bộ nhớ, tránh việc phải viết logic suy diễn phức tạp trực tiếp bằng Cypher.

```

1 def rule_cascading_parent_failure(facts):
2     candidates = []
3     devices, children_of = facts["devices"], facts["children_of"]
4     alarms_by_device = facts["alarms_by_device"]
5     for dev_id in devices:
6         if not _has_alarm_of_type(alarms_by_device, dev_id,
7                                   LINK_FAILURE_ALARM_TYPES):
8             continue
9         children = children_of.get(dev_id, [])
10        if not children:
11            continue
12        if all(_has_alarm_of_type(alarms_by_device, c,
13                                  SIGNAL_LOSS_ALARM_TYPES)
14              for c in children):
15            candidates.append(RuleCandidate(
16                device_id=dev_id, confidence=0.95,
17                rule_id="R1_cascading_parent_failure", ...))
18    return candidates

```

Listing 4.2: Luật R1 – cascading parent failure (src/rules/rule_engine.py)

4.2 Pipeline GraphRAG

Pipeline GraphRAG (src/graphrag/pipeline.py) gồm ba bước tuần tự, cài đặt tương ứng trong ba mô-đun:

- (1) **Retriever** (`retriever.py`): xác định *seed entities* (thiết bị có alarm/KPI thuộc sự cố đang xét), sau đó truy vấn Cypher để lấy tiểu đồ thị trong bán kính 2 bước nhảy (`DEPENDS_ON*0..2`) quanh các seed entities, gồm cả alarm/KPI/service/sự cố lịch sử liên quan.
- (2) **Context builder** (`context_builder.py`): chuyển tiểu đồ thị (dạng cấu trúc Python) thành văn bản tiếng Việt có cấu trúc rõ ràng (theo từng mục: thiết bị liên quan, cảnh báo hiện tại, KPI bất thường, dịch vụ bị ảnh hưởng, sự cố lịch sử tương tự).
- (3) **LLM client** (`llm_client.py`): gửi văn bản ngữ cảnh cho một LLM và yêu cầu trả về JSON gồm `root_cause_device`, `confidence`, `explanation`.

Đoạn mã 4.3 là truy vấn Cypher truy xuất tiểu đồ thị đa bước — điểm mấu chốt để GraphRAG có thể suy luận trên các mẫu hình lan truyền nhiều tầng thay vì chỉ nhìn một thiết bị đơn lẻ.

```

1 MATCH (seed:Device) WHERE seed.id IN $seed_ids
2 MATCH (seed)-[:DEPENDS_ON*0..2]-(neighbor:Device)
3 OPTIONAL MATCH (neighbor)-[:DEPENDS_ON]->(parent:Device)
4 OPTIONAL MATCH (neighbor)-[:BELONGS_TO_SITE]->(site:Site)
5 RETURN neighbor.id AS id, neighbor.type AS type, neighbor.name AS name,
6        parent.id AS parent_id, site.name AS site_name

```

Listing 4.3: Truy vấn Cypher truy xuất tiểu đồ thị đa bước

Thiết kế pluggable cho LLM. Lớp `LLMClient` trừu tượng được cài đặt bởi bốn lớp con: `MockLLM` (mặc định, không cần API key, suy luận offline bằng cách so khớp theo hệ số Jaccard với sự cố lịch sử kết hợp suy luận cấu trúc trên đồ thị), `AnthropicLLM`, `OpenAILLM` và `GroqLLM`. Việc chuyển đổi giữa các nhà cung cấp chỉ cần đổi biến môi trường `LLM_PROVIDER`, không cần sửa mã nguồn pipeline — một ví dụ áp dụng nguyên lý tách biệt giao diện (interface) khỏi cài đặt cụ thể.

Trong quá trình tích hợp LLM thật (Groq, mô hình `llama-3.3-70b-versatile`), nhóm phát hiện LLM thường bọc câu trả lời JSON trong đoạn văn bản suy luận và/hoặc khối markdown ````json` thay vì trả JSON thuần — khác với giả định ban đầu. Bộ phân tích JSON được viết lại theo hướng bền vững hơn: tìm khối ````json ... ```` bất kỳ vị trí nào trong văn bản, nếu không có thì tìm cặp dấu ngoặc `{...}` ngoài cùng, mới thử phân tích toàn bộ chuỗi.

4.3 Module tổng hợp và cơ chế kiểm chứng chéo

Module `src/synthesis/combiner.py` nhận đầu vào là danh sách ứng viên từ Rule Engine và một kết quả duy nhất từ GraphRAG, gộp theo thiết bị, rồi:

- Nếu một thiết bị được *cả hai* nguồn cùng đề xuất (đồng thuận), độ tin cậy được kết hợp bằng công thức (2.1) rồi cộng thêm một hệ số thưởng `CROSS_VALIDATION_BOOST = 0.1` (giới hạn tối đa 0.99) — thể hiện tinh thần ”hai nguồn độc lập cùng đồng ý thì đáng tin hơn một nguồn”.
- Nếu một thiết bị chỉ được GraphRAG đề xuất mà không có luật nào xác nhận, độ tin cậy tự báo cáo của LLM bị **chiết khấu** bằng hệ số `UNCORROBORATED_GRAPHRAG_DISCOUNT = 0.7` trước khi xếp hạng.

Hệ số chiết khấu ở gạch đầu dòng thứ hai **không** có trong thiết kế ban đầu, mà được thêm vào sau một phát hiện thực nghiệm thật khi thử nghiệm với Groq (trình bày chi tiết ở Mục 7.4): một LLM thật có thể tự báo cáo độ tin cậy cao dù trả lời sai, và nếu không chiết khấu, câu trả lời sai đó có thể lấn át một ứng viên đúng nhưng chỉ có Rule Engine ủng hộ. Đây là một minh chứng thực nghiệm cụ thể cho luận điểm lý thuyết ở Mục 2.4: suy diễn dựa luật, vì là tri thức đã được kiểm chứng một cách tường minh, cần được ưu tiên hơn một phỏng đoán đơn lẻ, không được xác nhận, của một mô hình thống kê.

```

1 effective_graphrag_conf = graphrag_conf
2 if graphrag_conf is not None and rule_conf is None:
3     effective_graphrag_conf = graphrag_conf *
4         UNCORROBORATED_GRAPHRAG_DISCOUNT
5
6 confs = [c for c in (rule_conf, effective_graphrag_conf) if c is not None]
7 combined_conf = _combine_certainty_factors(confs)
8 if agreement:
9     combined_conf = min(MAX_CONFIDENCE, combined_conf +
10         CROSS_VALIDATION_BOOST)

```

Listing 4.4: Chiết khấu ứng viên GraphRAG không được luật nào xác nhận (`src/synthesis/combiner.py`)

Cuối cùng, hàm `suggest_action` tra một bảng mẫu câu hướng xử lý theo loại thiết bị (`transmission`, `router`, `olt`, `gnodeb` cho viễn thông; `docker`, `db`, `os`, `osb` cho hạ tầng DejaVu), sinh khuyến nghị cụ thể thay vì một thông điệp chung chung.

Chương 5

Dữ liệu: hành trình tìm và tích hợp dữ liệu thật

5.1 Vì sao cần tìm dữ liệu thật

Đề cương ban đầu giới hạn phạm vi ở dữ liệu mô phỏng, vì dữ liệu vận hành thật của các nhà mạng luôn ở diện bảo mật. Tuy nhiên, để tăng tính thuyết phục của phương pháp, nhóm đã thử tìm các nguồn dữ liệu công khai, thật, có thể dùng để kiểm chứng độc lập – không phải để thay thế tập dữ liệu viển thông mô phỏng, mà để trả lời câu hỏi: *phương pháp có generalize được ra ngoài dữ liệu tự tạo hay không?*

5.2 Ba nguồn được khảo sát

Bảng 5.1 tóm tắt ba nguồn dữ liệu thật đã khảo sát và kết quả xác minh thực tế (không chỉ dựa trên tên/mô tả, mà đã thử tải trực tiếp).

Bảng 5.1: Ba nguồn dữ liệu thật đã khảo sát

Nguồn	Mô tả	Kết quả xác minh
NetRCA [3]	Alarm/KPI mạng 5G thật kèm causal graph, từ ICASSP 2022 AIOps Challenge – khớp nhất với đúng bài toán	Không dùng được: trang challenge không phân giải được DNS, không tìm thấy bản sao dữ liệu ở nơi khác
TeleLogs [4]	Dữ liệu RF tổng hợp mô phỏng UE lái xe qua vùng phủ 5G, host trên Hugging Face (netop/TeleLogs)	Không dùng được: xác nhận qua API là gated: manual (phải xin quyền, chờ duyệt thủ công) và chính dataset card cấm redistribute
NetManAIOps/DejaVu [5]	Dữ liệu lỗi hạ tầng IT-ops/microservice thật (Docker/DB/OS), kèm đồ thị phụ thuộc thật và ground-truth root cause, từ Zenodo	Dùng được: tải trực tiếp không cần đăng nhập, giấy phép CC-BY 4.0 cho phép redistribute một tập con

Hành trình này bản thân nó là một minh họa thực tế cho một vấn đề thường gặp khi làm việc với dữ liệu AIOps/viễn thông: dữ liệu vận hành thật gần như luôn bị giới hạn bởi hạ tầng không ổn định hoặc rào cản pháp lý/quyền truy cập, khiến các nghiên cứu buộc phải dựa vào dữ liệu mô phỏng hoặc dữ liệu thật từ miền lân cận.

5.3 Dữ liệu NetManAIOps/DejaVu

Bộ dữ liệu được công bố kèm bài báo tại ESEC/FSE 2022 [5] (Zenodo, DOI 10.5281/zenodo.6955909, giấy phép CC-BY 4.0). Nhóm viết một script tái tạo được (scripts/fetch_dejavu.py) để tải và phân tích hai tệp gốc:

- graph.yml: mô tả đồ thị phụ thuộc thật gồm 17 thiết bị – 3 database (db), 8 container (docker), 4 máy chủ vật lý (os) và 2 nút Online Service Bus (osb) – với 24 quan hệ gọi dịch vụ (*call edges*) và 8 quan hệ triển khai (container chạy trên máy chủ nào). Đây chính là cấu trúc DAG đa-cha đã thúc đẩy tổng quát hoá ở Mục 3.3.
- faults.csv: 78 bản ghi sự cố thật, mỗi bản ghi gồm loại lỗi (CPU fault, network delay, network loss, db connection limit, db close), thiết bị bị ảnh hưởng và **ground-truth root cause**. Toàn bộ 78 sự cố đều là lỗi đơn-thiết-bị (nguyên nhân luôn chính là thiết bị

phát sinh lỗi), không có trường hợp lan truyền qua nhiều thiết bị – khác với 5 kịch bản viễn thông vốn được thiết kế riêng để kiểm tra suy luận lan truyền (cascade).

Vì vậy, 78 sự cố thật của DejaVu chủ yếu kiểm tra khả năng nhận diện đúng lỗi khu trú (luật R2) trên một topology thật, phức tạp, không phải khả năng suy luận cascade – điều này được nêu rõ và không bị ”làm mờ” trong báo cáo đánh giá (Chương 7).

Lưu ý về tính trung thực khoa học: DejaVu là dữ liệu hạ tầng IT-ops/microservice thật, **không phải** dữ liệu viễn thông. Nó được dùng vì có cùng bản chất đồ thị-lý-thuyết (đồ thị phụ thuộc + lan truyền lỗi + định vị nguyên nhân gốc) với bài toán viễn thông, nên là một phép kiểm tra tổng quát hoá hợp lệ của *phương pháp*, chứ không phải một minh chứng về hiệu năng trên dữ liệu viễn thông thật.

Chương 6

Giao diện demo

Để đáp ứng yêu cầu "giao diện demo dạng web app đơn giản" của đề cương, nhóm xây dựng một ứng dụng web bằng Streamlit (app.py), gồm hai tab:

- **Tab "Chẩn đoán trực tiếp"**: người dùng chọn một trong hai bộ dữ liệu (viễn thông mô phỏng hoặc DejaVu thật), chọn một sự cố cụ thể, bấm nút để chạy toàn bộ pipeline. Kết quả hiển thị gồm: bảng ứng viên từ Rule Engine kèm luật kích hoạt, chẩn đoán của GraphRAG kèm độ tin cậy và giải thích, **sơ đồ đồ thị con trực quan** (thiết bị là node, quan hệ DEPENDS_ON là cạnh có hướng, tô màu theo trạng thái: đỏ = đang có alarm, cam = chỉ có KPI bất thường, vàng/ngôi sao = nguyên nhân gốc rễ được chẩn đoán), và khuyến nghị tổng hợp cuối cùng kèm hướng xử lý cụ thể.
- **Tab "Đánh giá tổng thể"**: chạy đánh giá hàng loạt trên một trong hai bộ dữ liệu, hiển thị biểu đồ cột so sánh độ chính xác giữa ba phương án (rule-only / GraphRAG-only / combined) cùng bảng chi tiết từng trường hợp.

Sơ đồ đồ thị con được cài đặt bằng thư viện pyvis (dựng trên vis-network.js), toàn bộ mã JavaScript được nhúng trực tiếp vào HTML (cdn_resources="in_line") để nhúng an toàn vào iframe của Streamlit mà không phụ thuộc tệp tĩnh bên ngoài.

Chương 7

Kết quả thực nghiệm và đánh giá

7.1 Phương pháp đánh giá

Theo đúng yêu cầu của đề cương, hệ thống được đánh giá bằng cách so sánh độ chính xác top-1 (thiết bị được xếp hạng cao nhất có trùng với ground-truth root cause hay không) giữa ba phương án: chỉ dùng Rule Engine, chỉ dùng GraphRAG, và kết hợp cả hai qua module tổng hợp. Đánh giá được thực hiện hai lần độc lập: một lần với MockLLM (xác định, không tốn chi phí, dùng để đánh giá số lượng lớn 78 sự cố DejaVu), và một lần với LLM thật (Groq) trên tập viển thông để kiểm tra hành vi trong điều kiện thực tế.

7.2 Kết quả trên tập viển thông mô phỏng

Bảng 7.1: Kết quả trên 5 kịch bản viển thông (MockLLM)

Kịch bản	Ground truth	Rule-only	GraphRAG-only	Combined
SCN-01	TRANS-CORE-1	✓	✓	✓
SCN-02	RTR-EDGE-2	✓	✓	✓
SCN-03	RTR-EDGE-1	✓	✓	✓
SCN-04	GNB-A2	✓	✓	✓
SCN-05	OLT-B1	✓	✓	✓
Độ chính xác		100% (5/5)	100% (5/5)	100% (5/5)

7.3 Kết quả trên 78 sự cố thật DejaVu

Bảng 7.2: Kết quả trên 78 sự cố thật (MockLLM)

Phương án	Độ chính xác
Rule-only	100% (78/78)
GraphRAG-only	100% (78/78)
Combined	100% (78/78)

Kết quả 100% trên cả hai tập với MockLLM cần được diễn giải cẩn trọng: vì toàn bộ sự cố DejaVu là lỗi đơn-điểm, và luật R2 (isolated device fault) được thiết kế chính xác cho mẫu hình này, kết quả cao là hệ quả tự nhiên của việc thiết kế luật đúng bản chất bài toán, không phải bằng chứng cho thấy hệ thống ”học” được từ dữ liệu DejaVu (hệ thống không có bước huấn luyện). Giá trị của kết quả này nằm ở chỗ: 4 luật và cơ chế MockLLM được xây dựng *trước và độc lập* với việc quan sát dữ liệu DejaVu, và vẫn generalize đúng sang một topology DAG đa-cha thật, phức tạp hơn hẳn tập viển thông tự tạo.

7.4 Thử nghiệm với LLM thật và phát hiện về hiệu chỉnh độ tin cậy

Khi thay MockLLM bằng GroqLLM (mô hình llama-3.3-70b-versatile, gọi API thật) trên 5 kịch bản viển thông, GraphRAG-only đạt 4/5 (80%): kịch bản SCN-05 (suy giảm KPI dần, không có alarm cứng) bị LLM chẩn đoán sai thành RTR-EDGE-2 thay vì OLT-B1 đúng. Đáng chú ý hơn, ở lần thử nghiệm đầu tiên (trước khi thêm cơ chế chiết khấu ở Mục 4.3), module tổng hợp **cũng cho kết quả sai theo GraphRAG**, vì độ tin cậy LLM tự báo cáo cho câu trả lời sai lại cao hơn độ tin cậy của ứng viên đúng từ Rule Engine (0.65).

Đây là một phát hiện thực nghiệm quan trọng, không phải lỗi cài đặt: LLM thật, khác với MockLLM (vốn được lập trình để trả về độ tin cậy có hiệu chỉnh), không đảm bảo độ tin cậy tự báo cáo phản ánh đúng xác suất chính xác thực tế – một vấn đề đã được ghi nhận rộng rãi trong tài liệu về hiệu chỉnh độ tin cậy (confidence calibration) của LLM [2]. Sau khi bổ sung cơ chế chiết khấu ứng viên GraphRAG không được luật xác nhận (Đoạn mã 4.4), kết quả *combined* khôi phục về 5/5 (100%) mà không làm giảm độ chính xác ở bất kỳ trường hợp nào khác (đã chạy lại toàn bộ 5 kịch bản viển thông và 78 sự cố DejaVu để xác nhận). Kết quả này minh hoạ cụ thể, bằng số liệu thực nghiệm, cho luận điểm lý thuyết đã nêu ở Mục 2.4 và trong đề cương ban đầu: suy diễn dựa luật đóng vai trò kiểm chứng chéo không thể thay thế cho suy luận của LLM, ngay cả khi LLM đã có ngữ cảnh được truy xuất chính xác từ Knowledge Graph.

7.5 Phân tích lỗi có hệ thống trên mẫu mở rộng (Error Analysis)

Kết quả ở Mục 7.4 đặt ra một câu hỏi: liệu sai lầm của GroqLLM ở SCN-05 là ngẫu nhiên, hay phản ánh một xu hướng suy luận sai có hệ thống? Vì độ chính xác top-1 đã bão hoà ở 100% trên cả hai tập dữ liệu khi dùng MockLLM, chạy toàn bộ 78 sự cố DejaVu qua LLM thật sẽ tốn thời gian/quota API mà không chắc sinh thêm thông tin định lượng mới – một đánh đổi (trade-off) kinh điển khi một độ đo đã chạm trần (ceiling effect): thay vì thêm số liệu tổng hợp, giá trị nằm ở phân tích định tính các trường hợp sai. Một mẫu 20 sự cố được chọn có chủ đích để tối đa hoá khả năng LLM mắc lỗi: toàn bộ 5 kịch bản viển thông, cộng 15 sự cố DejaVu được chọn theo **độ khó cấu trúc** – số thiết bị cha (DEPENDS_ON) của thiết bị bị cảnh báo, vì thiết bị càng nhiều cha thì càng có nhiều ”nghị phạm giả” khiến suy luận dựa trên ngữ cảnh dễ đi lạc hướng. Cụ thể: 12 sự cố trên bốn thiết bị ”khó nhất” (docker_001 đến docker_004, mỗi thiết bị có 5 thiết bị cha), phủ đủ tổ hợp (thiết bị, loại lỗi) có thể xảy ra trên nhóm này; và 3 sự cố trên nhóm thiết bị có 2 thiết bị cha, để có điểm so sánh phổ độ khó.

Bảng 7.3 so sánh MockLLM và GroqLLM trên đúng mẫu 20 sự cố này.

Bảng 7.3: So sánh MockLLM và GroqLLM (LLM thật) trên mẫu 20 sự cố chọn lọc theo độ khó

Phương án	MockLLM	GroqLLM (LLM thật)
Rule-only	100% (20/20)	100% (20/20)
GraphRAG-only	100% (20/20)	45% (9/20)
Combined	100% (20/20)	100% (20/20)

Ba quan sát quan trọng:

- (1) Rule Engine không đổi (100% cả hai cột) – đúng như kỳ vọng, vì nó không phụ thuộc LLM.
- (2) GraphRAG-only **sụt từ 100% xuống 45%** khi chuyển từ MockLLM sang LLM thật, trên đúng mẫu được chọn để ”làm khó” nó – một khoảng cách lớn hơn nhiều so với con số 80% quan sát được ở SCN-05 đơn lẻ (Mục 7.4), cho thấy đây không phải hiện tượng hiếm.
- (3) Combined vẫn giữ nguyên **100%** – toàn bộ 11/11 trường hợp GraphRAG trả lời sai đều được module tổng hợp sửa đúng nhờ cơ chế chiết khấu (Mục 4.3).

Một khuôn mẫu lỗi duy nhất, không phải nhiễu ngẫu nhiên. Xem xét chi tiết cả 11 trường hợp GraphRAG trả lời sai (Bảng 7.4) cho thấy chúng đều theo **đúng một khuôn mẫu**: LLM luôn chọn một *thiết bị cha* (db_XXX hoặc os_XXX) của thiết bị thực sự bị lỗi làm nguyên nhân

gốc rễ, với lý do ”thiết bị cha này được nhiều thiết bị khác cùng phụ thuộc, nên có thể là nguyên nhân chung” – trong khi thiết bị cha đó **không hề có alarm hay bằng chứng bất thường nào** trong ngữ cảnh được cung cấp. Đây là một dạng suy luận quy nạp thiếu căn cứ (unwarranted generalization): LLM đánh đồng ”được nhiều thiết bị phụ thuộc” với ”có khả năng là nguyên nhân” – một suy luận hợp lý *trong trường hợp cascade thật* (đúng như luật R3 của Rule Engine mã hoá tường minh, Bảng 4.1) nhưng sai trong các trường hợp này, vì toàn bộ 78 sự cố DejaVu đều là lỗi đơn-thiết-bị (Mục 5) – không có thiết bị nào khác thực sự bị ảnh hưởng để biện minh cho suy luận đó.

Bảng 7.4: Toàn bộ 11 trường hợp GraphRAG (LLM thật) trả lời sai trong mẫu 20 sự cố

ID	Ground truth	GraphRAG đoán	Tóm tắt lý do LLM đưa ra (đều theo cùng một khuôn mẫu)
SCN-05	OLT-B1	RTR-EDGE-2	Không có alarm, chỉ KPI bất thường ở OLT-B1; nhầm sang case lịch sử liên quan router
DEJAVU-002	docker_002	db_009	”db_009 là thiết bị nhiều docker khác cùng phụ thuộc”
DEJAVU-003	docker_001	docker_007	Nhầm sang một thiết bị ngang hàng không liên quan trực tiếp
DEJAVU-004	docker_004	db_007	”db_007 là thiết bị chung của nhiều docker”
DEJAVU-006	docker_002	db_009	”db_009 là thiết bị chung cho nhiều docker”
DEJAVU-008	docker_003	os_019	”os_019 là thiết bị có thể ảnh hưởng đến docker_003”
DEJAVU-013	docker_001	db_003	Suy luận bắc cầu qua một thiết bị cha không liên quan trực tiếp
DEJAVU-029	docker_002	db_009	”db_009 được nhiều docker khác phụ thuộc”
DEJAVU-032	docker_004	db_007	”db_007 là thiết bị chung, không có thông tin nên nghi ngờ nó”
DEJAVU-010	docker_007	db_003	”db_003 cũng là thiết bị phụ thuộc của các docker khác”
DEJAVU-014	docker_005	db_003	”các thiết bị khác phụ thuộc db_003 cũng có thể bị ảnh hưởng”

Phát hiện này có ý nghĩa thực tiễn rõ ràng cho một hệ thống hỗ trợ NOC: nếu triển khai GraphRAG đơn lẻ (không có Rule Engine), hệ thống sẽ có xu hướng **hệ thống** (không ngẫu nhiên) đổ oan cho hạ tầng dùng chung (database, host) mỗi khi một dịch vụ đơn lẻ gặp sự cố – một sai lầm chẩn đoán tốn kém trong vận hành thực tế (điều tra nhầm hướng, có thể ảnh

hướng đây chuyển đến các dịch vụ khác đang chạy tốt trên cùng hạ tầng dùng chung). Việc Rule Engine sửa đúng toàn bộ 11/11 trường hợp này không phải may mắn: luật R2 (isolated device fault, Bảng 4.1) kiểm tra chính xác điều kiện LLM đã bỏ qua – liệu thiết bị cha có bằng chứng bất thường hay không, chứ không suy luận từ cấu trúc phụ thuộc đơn thuần. Toàn bộ dữ liệu thô (bao gồm nguyên văn giải thích của LLM cho từng trường hợp) được lưu tại `report/real_llm_sample_results.json`, sinh ra bởi `src/evaluation/evaluate_real_llm_sample.py`.

7.6 Kiểm thử phần mềm

Bộ kiểm thử tự động (pytest) gồm 14 test case, bao trùm: logic từng luật của Rule Engine, công thức kết hợp certainty factor, bộ phân tích ngữ cảnh (context_builder), suy luận của MockLLM (case-based matching và suy luận cấu trúc), cơ chế chiết khấu ở module tổng hợp, và tính toàn vẹn của dữ liệu DejaVu đã tải về. Toàn bộ test không phụ thuộc Neo4j đang chạy hay không (dữ kiện được dựng trực tiếp bằng cấu trúc Python), giúp chạy nhanh (dưới 0.1 giây) và có thể tích hợp vào quy trình kiểm tra liên tục.

Chương 8

Kết luận và hướng phát triển

8.1 Kết luận

Báo cáo đã trình bày một hệ thống hoàn chỉnh, cài đặt được và kiểm chứng được, minh hoạ ba kỹ thuật biểu diễn và suy diễn tri thức của học phần trên cùng một bài toán thực tế: Knowledge Graph để biểu diễn tường minh cấu trúc phụ thuộc, suy diễn dựa luật theo mô hình production system với cơ chế certainty factor kiểu MYCIN, và GraphRAG để khai thác LLM một cách có kiểm soát. Hai đóng góp thực nghiệm đáng chú ý ngoài phạm vi tối thiểu của đề cương ban đầu là: (1) tổng quát hoá kiến trúc sang đồ thị phụ thuộc đa-cha và kiểm chứng độc lập trên 78 sự cố hạ tầng thật (NetManAIOps/DejaVu), và (2) phát hiện và khắc phục vấn đề hiệu chỉnh độ tin cậy khi tích hợp LLM thật, cho thấy giá trị cụ thể, đo lường được của cơ chế kiểm chứng chéo giữa luật và LLM.

8.2 Hướng phát triển

- Mở rộng bộ luật để xử lý các mẫu hình lan truyền phức tạp hơn (nhiều tầng cascade đồng thời, sự cố gián đoạn/không liên tục).
- Đánh giá thêm với các LLM khác (mô hình lớn hơn, hoặc mô hình được fine-tune riêng cho domain vận hành mạng) để so sánh mức độ hiệu chỉnh độ tin cậy.
- Xây dựng cơ chế học/cập nhật trọng số luật từ phản hồi của kỹ sư vận hành (human-in-the-loop), thay vì trọng số cố định như hiện tại.
- Nếu có điều kiện tiếp cận dữ liệu vận hành thật của một nhà mạng (qua thoả thuận bảo mật), kiểm chứng lại toàn bộ phương pháp trên đúng miền viễn thông với dữ liệu thật.

Tài liệu tham khảo

- [1] E. H. Shortliffe and B. G. Buchanan, *A model of inexact reasoning in medicine*, Mathematical Biosciences, 23(3-4):351–379, 1975.
- [2] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, *On Calibration of Modern Neural Networks*, Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.
- [3] Signal Processing Society, IEEE, *Root Cause Analysis for Wireless Network Faults Localization (NetRCA) – ICASSP 2022 AIOps Challenge*. <https://signalprocessingsociety.org/publications-resources/data-challenges/root-cause-analysis-wireless-network-faults-localization>
- [4] NetOp Team, Huawei Paris Research Center, *TeleLogs: A Benchmark for Large Language Models in Telecom Data Analysis*, Hugging Face Datasets, <https://huggingface.co/datasets/netop/TeleLogs>, 2025.
- [5] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, D. Pei, *Actionable and Interpretable Fault Localization for Recurring Failures in Online Service Systems*, Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2022. Dữ liệu: Zenodo, DOI: 10.5281/zenodo.6955909, giấy phép CC-BY 4.0.
- [6] Neo4j, Inc., *Neo4j Graph Database Documentation*, <https://neo4j.com/docs/>.